The National Higher School of Artificial Intelligence
Natural Language Processing
Final Exam 28/11/2024
Duration: 1 hour 30 minutes

Exercise 1		**/ 7.5**
Exercise 2		**/ 13**
TOTAL		**/20.5**
		/20

Exercise 1: **7.5 points**

In this exercise, you will build a modified version of the multinomial Naive Bayes classifier that uses a bigram language model for sentiment classification.

Training Data

- **Positive (pos)**: "Liking movie, good acting!!"
- **Positive (pos)**: "Liked acting, really good!"
- **Negative (neg)**: "Really bad! Bad acting!!"
- **Negative (neg)**: "Played badly, real bad!! Hating movie!"

Part 1 - Naive Bayes Formula **(0.25+0.75 = 1 point)**

a. Write the general Naive Bayes formula (the base formula, without using the log-linear form).

$$P(class|text) = P(class) * \prod P(word|class)$$

b. Write a modified version of the formula that incorporates a bigram language model (still without using the log-linear form).

$$P(class|text) = P(class) * \prod P(word_i|word_{i-1}, class)$$

Part 2 - Sentiment Classification: **(1+1+3 = 5 points)**

Given the following assumptions:

- Preprocessing will include **word tokenization**, **punctuation removal**, **word normalization**, and **lemmatization**. Special tokens that are necessary for the bigram language model will also be added.
- **Add-1 smoothing** (Laplace smoothing) will be applied for the language model.

a. Provide the training sentences after preprocessing.

Preprocessing results (train sents):

Positive (pos): <s> like movie good act <\s>
Positive (pos): <s> like act real good <\s>
Negative (neg): <s> real bad bad act <\s>
Negative (neg): <s> play bad real bad hate movie <\s>

b. Provide the add-1 smoothing equation that will be used for the language model as well as the content of the vocabulary V that will be used in it.

Complete Vocabulary (10 tokens):

V = ['</s>', '<s>', 'act', 'bad', 'good', 'hate', 'like', 'movie', 'play', 'real']
|V| = 10

$$P(word_i|word_{i-1},class) = (count((word_{i-1},word_i),class) + 1) / (count(word_{i-1},class) + |V|)$$

c. Using the assumptions above, and the modified formula from question 1.b, classify the sentiment of the following test sentences. Show your calculations in detail.

Test Sentences:

- **Sent 1**: "Liked acting, bad!!!"
- **Sent 2**: "Really bad!!! liking movie, good!"

Preprocessing results (test sents):
Sent 1: <s> like act bad <\s>
Sent 2: <s> real bad like movie good <\s>

==========
Prior Probabilities
==========
P(Positive) = 2/4 = 0.5000
P(Negative) = 2/4 = 0.5000

==========
Test Sentence 1:
Classifying sentence: '<s> like act bad </s>'
==========

Class 'Positive':
Prior probability: P(Positive) = 0.500000
Bigram probabilities:
P(like|<s>, Positive) = (2+1.0)/(2+10) = 0.250000
P(act|like, Positive) = (1+1.0)/(2+10) = 0.166667
P(bad|act, Positive) = (0+1.0)/(2+10) = 0.083333
P(</s>|bad, Positive) = (0+1.0)/(0+10) = 0.100000
Final probability:
P(Positive|Sentence1) = 0.5 * 0.25 * 0.167 * 0.083 * 0.1 = **0.0001736111**

Class 'Negative':
Prior probability: P(Negative) = 0.500000
Bigram probabilities:
P(like|<s>, Negative) = (0+1.0)/(2+1.0*10) = 0.083333
P(act|like, Negative) = (0+1.0)/(0+1.0*10) = 0.100000
P(bad|act, Negative) = (0+1.0)/(1+1.0*10) = 0.090909
P(</s>|bad, Negative) = (0+1.0)/(4+1.0*10) = 0.071429
Final probability:
P(Negative|Sentence1) = 0.5 * 0.083 * 0.1 * 0.091 * 0.071 = **0.0000270563**

➔**Predicted class: Positive**

==========
Test Sentence 2:
Classifying sentence: '<s> real bad like movie good </s>'
==========

Class 'Positive':
Prior probability: P(Positive) = 0.500000

Bigram probabilities:
P(real|<s>, Positive) = (0+1.0)/(2+1.0*10) = 0.083333
P(bad|real, Positive) = (0+1.0)/(1+1.0*10) = 0.090909
P(like|bad, Positive) = (0+1.0)/(0+1.0*10) = 0.100000
P(movie|like, Positive) = (1+1.0)/(2+1.0*10) = 0.166667
P(good|movie, Positive) = (1+1.0)/(1+1.0*10) = 0.181818
P(</s>|good, Positive) = (1+1.0)/(2+1.0*10) = 0.166667
Final probability:
P(Positive|Sentence2) = 0.5 * 0.083 * 0.091 * 0.1 * 0.167 * 0.182 * 0.167 = **0.0000019131**

Class 'Negative':
Prior probability: P(Negative) = 0.500000
Bigram probabilities:
P(real|<s>, Negative) = (1+1.0)/(2+1.0*10) = 0.166667
P(bad|real, Negative) = (2+1.0)/(2+1.0*10) = 0.250000
P(like|bad, Negative) = (0+1.0)/(4+1.0*10) = 0.071429
P(movie|like, Negative) = (0+1.0)/(0+1.0*10) = 0.100000
P(good|movie, Negative) = (0+1.0)/(1+1.0*10) = 0.090909
P(</s>|good, Negative) = (0+1.0)/(0+1.0*10) = 0.100000
Final probability:
P(Negative|Sentence2) = 0.5 * 0.167 * 0.25 * 0.0714 * 0.1 * 0.091 * 0.1= **0.0000013528**

➔**Predicted class: Positive**

Part 3 – Reflection **(0.5+0.5+0.5 = 1.5 points)**

a. Do you think lemmatization was useful in this exercise? Justify your answer.

Lemmatization: Lemmatization transforms words like 'playing,' 'played,' and 'plays' in our dataset into their base form, 'play.' This process reduces feature sparsity (i.e., the size of the vocabulary) by treating different forms of the same word as a single entity. By doing so, it improves probability estimates, which is particularly advantageous for small datasets like ours, where variations in word forms could otherwise result in a large and sparse vocabulary. However, as the dataset size grows, the impact of lemmatization diminishes because the model has sufficient data to learn relationships between different word forms.

b. Do you think the bigram language model was useful in this exercise? Justify your answer.

Bigram Language Model: The bigram model was not particularly useful in our exercise because we had limited instances in our training data. As a result, many bigrams in the test set were unseen during training, leading to poor performance and sparse probability estimates.

c. What would happen if we had used a 4-gram language model instead? Justify your reasoning.

4-gram Language Model: A 4-gram model would perform even worse in our case due to extreme data sparsity, the limited size of our training data. Higher-order n-grams, such as 4-grams, require significantly more data to capture meaningful patterns, and without sufficient training examples, these models tend to underperform.

Exercise 2: **13 points**

In this exercise, you will use a modified TF-IDF formula to represent a set of documents, considering both unigram and bigram features.

Training Data

You are given the following set of documents:

- **Document 1**: "He loves music."
- **Document 2**: "He love programming."
- **Document 3**: "He loves programing and traveling!"

Part 1 - Standard TF-IDF Formula **(1 + 1 = 2 point)**

a) Write the general TF-IDF formula (the base formula, without using the log) stating what each factor represents.

TF-IDF(t,d)=TF(t,d)·IDF(t)

Where:

- TF(t,d): Term frequency of term t in document d which is equal to count(t,d).
- IDF(t): $\frac{N}{DF(t)}$ where N is the number of documents in the training set. DF(t): Number of documents containing t.

b) Write a modified version of the TF-IDF formula that applies $1+\log_{10}(TF)$ to the term frequency and $[1+\log_{10}(IDF)]^2$ to the inverse document frequency. Explain the goal of these adjustments. There's no need to add 1 to TF or IDF inside the logarithm, as we assume that TF-IDF will only be calculated for terms that exist in the vocabulary (0 otherwise).

Modified-TF-IDF(t,d)=[1+log10(TF(t,d))] * $[1+\log10(IDF(t))]^2$

Adjustments:

- 1+log10(TF): The logarithm reduces the effect of large term frequencies, while the addition of 1 ensures non-zero values.
- $[1+\log10(IDF)]^2$: Squaring the IDF increases its impact, giving more importance to terms that characterize a specific document.

Part 2 - Modified TF-IDF **(1.5 + 1 + 3.5 = 6 points)**

Given the following assumptions:

- Preprocessing will include **word tokenization, punctuation removal, word normalization, pronouns and connectors removal, and lemmatization**.
- The TF-IDF will consider both unigrams and bigrams as features.
- TF-IDF will use the formula from 1.b.

a. Provide the resulting training documents after preprocessing.

Document 1: "love music"
Document 2: "love program"
Document 3: "love program travel"

b. Build the vocabulary from the training data and provide its size.

The vocabulary includes both unigrams and bigrams from the preprocessed training set:

- Vocabulary = {"love", "music", "program", "travel", "love music", "love program", "program travel"}
- Vocabulary size: 7 terms.

c. Compute the IDF scores for each entry in the vocabulary for each document (fill the below matrix).

Vocabulary Terms	DF	IDF ($\frac{N}{DF}$)	$[1+\log_{10}(IDF)]^2$
love	3	$\frac{3}{3}=1$	1
music	1	$\frac{3}{1}=3$	2.18
program	2	$\frac{3}{2}=1.5$	1.38
travel	1	$\frac{3}{1}=3$	2.18
love music	1	$\frac{3}{1}=3$	2.18
love program	2	$\frac{3}{2}=1.5$	1.38
program travel	1	$\frac{3}{1}=3$	2.18

Part 3 - Query Representation and Similarity **(2 + 3 = 5 points)**

a) For the given test query: "**He loves music, programming, and traveling.**", represent it along with all the training documents using the TF-IDF matrix below.

- Begin by listing all the terms (entries) in the TF-IDF vocabulary as the first row of the matrix.
- For each document (including the test query), calculate and populate the TF-IDF values for each term in the vocabulary.
- The result will be a vector representation of each document, including the test query.

Preprocessed Query: "love music program travel"

The trick here is to notice that since each term t in the query exist exactly once, then:

$1+\log_{10}(TF(t, D)) = 1$, if t in D

0 otherwise

⇨ This means that if a term exists in D, its TF-IDF score will be exactly its IDF, otherwise it will be 0. Meaning we don't need to calculate the TFs.
Query TF-IDF vector:

	love	music	program	travel	love music	love program	program travel
Q	1	2.18	1.38	2.18	2.18	0	2.18
D1	1	2.18	0	0	2.18	0	0
D2	1	0	1.38	0	0	1.38	0
D3	1	0	1.38	2.18	0	1.38	2.18

b) Write the cosine similarity equation that finds the similarity between two vectors. Use this equation to calculate the cosine similarity between the query vector and each training document vector. Rank the training documents based on their similarity to the query, from highest to lowest.

$$similarity(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \times \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Query Magnitude:

$$\|\vec{q}\| = \sqrt{1^2 + 2.18^2 + 1.38^2 + 2.18^2 + 2.18^2 + 0^2 + 2.18^2}$$

$$\|\vec{q}\| = \sqrt{1 + 4.75 + 1.90 + 4.75 + 4.75 + 0 + 4.75} = \sqrt{21.90} \approx 4.68$$

Document Similarities:

1. **Document 1**:
   - Dot Product:
     $(1 \cdot 1) + (2.18 \cdot 2.18) + (0 \cdot 0) + (0 \cdot 0) + (2.18 \cdot 2.18) + (0 \cdot 0) + (0 \cdot 0) = 1 + 4.75 + 4.75 = 10.50$
   - Magnitude:
     $\|\vec{d}_1\| = \sqrt{1^2 + 2.18^2 + 0^2 + 0^2 + 2.18^2 + 0^2 + 0^2} = \sqrt{1 + 4.75 + 4.75} = \sqrt{10.50} \approx 3.24$
   - Cosine Similarity:
     $$\frac{10.50}{4.68 \cdot 3.24} = \frac{10.50}{15.16} \approx 0.69$$
2. **Document 2**:
   - Dot Product:
     $(1 \cdot 1) + (2.18 \cdot 0) + (1.38 \cdot 1.38) + (0 \cdot 0) + (0 \cdot 0) + (0 \cdot 1.38) + (0 \cdot 0) = 1 + 1.90 = 2.90$
   - Magnitude:
     $\|\vec{d}_2\| = \sqrt{1^2 + 0^2 + 1.38^2 + 0^2 + 0^2 + 1.38^2 + 0^2} = \sqrt{1 + 1.90 + 1.90} = \sqrt{4.80} \approx 2.19$
   - Cosine Similarity:
     $$\frac{2.90}{4.68 \cdot 2.19} = \frac{2.90}{10.25} \approx 0.28$$
3. **Document 3**:
   - Dot Product:
     $(1 \cdot 1) + (2.18 \cdot 0) + (1.38 \cdot 1.38) + (2.18 \cdot 2.18) + (0 \cdot 0) + (0 \cdot 1.38) + (2.18 \cdot 2.18) = 1 + 1.90 + 4.75 + 4.75 = 12.40$
   - Magnitude:
     $\|\vec{d}_3\| = \sqrt{1^2 + 0^2 + 1.38^2 + 2.18^2 + 0^2 + 1.38^2 + 2.18^2} = \sqrt{1 + 1.90 + 4.75 + 1.90 + 4.75} = \sqrt{14.30} \approx 3.78$
   - Cosine Similarity:
     $$\frac{12.40}{4.68 \cdot 3.78} = \frac{12.40}{17.68} \approx 0.70$$